Revive Peptides Co: Whop purchase tracking, server side

Prepared 7 September 2026 for the Revive Peptides Co development team. Business `biz_b3aIz02MxBmvIg` / `revivepeptidesco.com`. **This document replaces section 4b of the first handover.** Everything else in that document, the diagnosis and the site wide pixel in section 4a, still stands.

1. Why this changed

The first handover fired the purchase from the browser, on the order confirmation page. That works for card orders and is the common pattern, but it is the wrong primary choice for **this** store, for a reason that is specific and checkable.

> **This store accepts Zelle and Same-Day Bank Payment as well as cards.** Those orders get marked paid **after the fact**, out of band, by a person or a webhook. When that happens there is **no browser anywhere**, so a confirmation page never renders and a browser pixel cannot fire. Every one of those sales would simply be missing, and the gap would look like weak ad performance rather than a measurement hole.

So the purchase event moves server side. The browser pixel stays, but its job is now top of funnel only.

2. The shape

what	where it runs	why
`page`, `view_content`, `add_to_cart`, `initiate_checkout`	browser pixel, site wide (section 4a of the first doc)	needs the visitor's browser, and sets the `_wuid` cookie the purchase is matched on
`whop_purchase`	**server side, PHP**	fires whenever the order becomes paid, browser present or not

> **Keep the site wide pixel.** It is not optional once the purchase moves server side, because it is what issues the `_wuid` visitor id. Without it the server side event still records the sale but has nothing to attribute it to, and the ads dashboard will show revenue with no campaign behind it.

3. The attribution link, which is the part that is easy to get wrong

The Whop pixel sets a cookie named `_wuid` (note the leading underscore, it is not `whop_wuid`). That value is the visitor id Whop matches the purchase back to an ad click with.

> **Read that cookie at CHECKOUT and store it on the order. Do not try to read it when the order becomes paid.** A Zelle order may be marked paid days later from wp-admin, where there is no customer browser and therefore no cookie. If you read it at send time it will be empty for exactly the orders this whole change exists to capture.

4. The request

Measured against the live API, not copied from documentation:

```
POST https://api.whop.com/api/v1/events
Content-Type: application/json

{
  "event_name": "whop_purchase",
  "company_id": "biz_b3aIz02MxBmvIg",
  "event_id":   "12345",
  "value":      87.19,
  "currency":   "usd",
  "url":        "https://revivepeptidesco.com/checkout/order-received/12345/",
  "user":    { "anonymous_id": "wuid_...", "email": "buyer@example.com" },
  "context": { "user_agent": "Mozilla/5.0 ..." }
}
```

Accepted fields, exactly: `event_name`, `company_id`, `custom_name` (accepted then ignored), `value`, `currency`, `url`, `event_id`, `user{}`, `context{}`. `user` accepts `anonymous_id`, `email`, `user_id`, `external_id`, `phone`, `name`. `context` accepts `user_agent`, `fbclid`, `gclid`, `ttclid`. Anything else is a 400, and nested validation is strict, so this list is measured rather than guessed.

No API key is required on this endpoint, and that is deliberate on Whop's part: it means no credential has to be stored on the store. (Control: an unauthenticated `GET` on the same family returns 401, so the acceptance is real behavior and not a blanket open API.)

`currency` must be lowercase. `event_id` is the dedup key and comes back as `<company>:<event_id>`.

5. The one that fails silently and permanently

> **On this server side endpoint the custom event name goes in `event_name`.** A `custom_name` you send is accepted and then **ignored**, and Whop derives `custom_name` from `event_name`. Measured on a live business:

sent	stored
`event_name: "whop_purchase"`, no custom_name	`pixel.custom` / custom_name **`whop_purchase`**, correct
`event_name: "pixel.custom"` + `custom_name: "whop_purchase"`	`pixel.custom` / custom_name **`pixel.custom`**, the name is LOST

The wrong shape returns a success response. Orders keep arriving, rows keep landing, and the only symptom is that a campaign optimizing on `whop_purchase` reads **zero forever**, which reads as bad creative rather than a broken field name. **Whop has no delete endpoint**, so every wrong row is permanent.

> **This rule is the exact opposite of the browser pixel**, where the name is the FIRST ARGUMENT to `whop.track()` and an `event_name` property is silently dropped. Two surfaces, inverted rules, both failing quietly. Know which one you are writing.

6. Test without touching the live business

Because there is no delete endpoint, validate your payload shape against a company id that does not exist:

```
company_id: "biz_zzzzzzzzzzzzz"
```

Field validation runs **before** company resolution, so a well formed payload returns **404** `"This Bot was not found"` and **creates nothing**, while a malformed one returns **400** naming the bad parameter. That gives you a free, side effect free way to prove the shape before you point it at the real business.

7. The implementation

Drop this in a site specific plugin, or in a WPCode PHP snippet set to run everywhere.

```
/**
 * Revive Peptides Co -> Whop server side purchase tracking.
 * Drop in a site specific plugin, or a WPCode PHP snippet run everywhere.
 */

const REVIVE_WHOP_COMPANY_ID = 'biz_b3aIz02MxBmvIg';

/**
 * STEP 1. Capture the Whop visitor id at checkout, while a browser is
 * still present. This is the whole attribution link. A Zelle order may be
 * marked paid days later with nobody in a browser, so it MUST be stored
 * on the order now, not read at send time.
 */
add_action( 'woocommerce_checkout_create_order', function ( $order ) {
    if ( ! empty( $_COOKIE['_wuid'] ) ) {
        $wuid = sanitize_text_field( wp_unslash( $_COOKIE['_wuid'] ) );
        $order->update_meta_data( '_whop_wuid', $wuid );
    }
    if ( ! empty( $_SERVER['HTTP_USER_AGENT'] ) ) {
        $ua = wp_unslash( $_SERVER['HTTP_USER_AGENT'] );
        $ua = substr( sanitize_text_field( $ua ), 0, 400 );
        $order->update_meta_data( '_whop_ua', $ua );
    }
}, 10, 1 );

/**
 * STEP 2. Send the purchase once, when the order actually becomes paid.
 * Three hooks on purpose: card payments fire payment_complete, while a
 * Zelle or bank order marked paid by hand in wp-admin fires only the
 * status transition.
 */
add_action( 'woocommerce_payment_complete',        'revive_whop_send', 10, 1 );
add_action( 'woocommerce_order_status_processing', 'revive_whop_send', 10, 1 );
add_action( 'woocommerce_order_status_completed',  'revive_whop_send', 10, 1 );

function revive_whop_send( $order_id ) {
    $order = wc_get_order( $order_id );
    if ( ! $order ) {
        return;
    }

    /* Fire once per order, ever. event_id is Whop's dedup key and
       there is no delete endpoint. */
    if ( $order->get_meta( '_whop_purchase_sent' ) ) {
        return;
    }

    /* The name goes in event_name, NOT custom_name. See section 5. */
    $payload = array(
        'event_name' => 'whop_purchase',
        'company_id' => REVIVE_WHOP_COMPANY_ID,
        'event_id'   => (string) $order->get_id(),
        'value'      => (float) $order->get_total(),
        'currency'   => strtolower( $order->get_currency() ),
        'url'        => $order->get_checkout_order_received_url(),
    );

    $user = array();
    $wuid = $order->get_meta( '_whop_wuid' );
```

```
    if ( $wuid ) {
        $user['anonymous_id'] = $wuid;
    }
    if ( $order->get_billing_email() ) {
        $user['email'] = $order->get_billing_email();
    }
    if ( $user ) {
        $payload['user'] = $user;
    }

    $ua = $order->get_meta( '_whop_ua' );
    if ( $ua ) {
        $payload['context'] = array( 'user_agent' => $ua );
    }

    $res = wp_remote_post( 'https://api.whop.com/api/v1/events', array(
        'timeout' => 15,
        'headers' => array( 'Content-Type' => 'application/json' ),
        'body'    => wp_json_encode( $payload ),
    ) );

    $code = is_wp_error( $res ) ? 0 : (int) wp_remote_retrieve_response_code( $res );

    /* Only mark sent on a real 2xx, so a transient Whop outage retries
       on the next status change instead of dropping the order forever. */
    if ( $code >= 200 && $code < 300 ) {
        $order->update_meta_data( '_whop_purchase_sent', 1 );
    } else {
        $why = is_wp_error( $res )
            ? $res->get_error_message()
            : wp_remote_retrieve_body( $res );
        $order->update_meta_data( '_whop_purchase_error', $code . ' ' . $why );
        $order->add_order_note(
            'Whop purchase event failed, HTTP ' . $code .
            '. Will retry on the next status change.'
        );
    }
    $order->save();
}
```

8. Notes on the choices above, since they are not arbitrary

1. **Three hooks, not one.** Card payments fire `woocommerce_payment_complete`. A Zelle or bank order marked paid by hand in wp-admin fires only the status transition, so `processing` and `completed` are needed too. The dedup guard makes the overlap harmless.
2. **It only marks itself sent on a 2xx.** If Whop is down, the order is left unmarked and retries on the next status change, instead of being silently dropped forever. Whop's events API had a multi hour outage on the day this was written, which is exactly the case this covers.
3. **The failure is written to an order note.** A silent failure on a money event is the worst outcome, so a failed send is visible in the order screen rather than only in a log.
4. **event_id is the real order id**, so a retry cannot double count.

9. How to verify

1. Place one real card order. Confirm the order gets `_whop_purchase_sent` in its meta and no failure note.
2. Read the event back and confirm it appears **exactly once** with `custom_name` equal to `whop_purchase`, the right value, and lowercase currency:

```
GET https://api.whop.com/api/v1/events?company_id=biz_b3aIz02MxBmvIg&from=<iso>&to=<iso>
```

`from` and `to` are both required and the range cannot exceed **30 days**, or you get a 400. That 400 is a payload error, not an auth error.

Then test the case this change exists for: place a Zelle order, leave it unpaid, and mark it paid manually in wp-admin. The event must appear then too. If it only works for cards, the capture in step 1 is being read at send time instead of at checkout.

> **One caution on timing.** On the day this was written Whop's events API was returning 500s and, worse, occasional **200s carrying an empty result set**. If you verify during an outage like that, a correct install can read as a failed one. Confirm the API is healthy before concluding the tracking is broken.

10. What is not claimed

The PHP above has been **parsed** and is syntactically valid, and the request payload has been **validated against the live API**. It has **not** been executed against this store, because that requires placing a real order on a live shop. Treat step 9 as required, not optional.